



Multi-modal RAG Implementation: FireGPT

Project report

Muhammad Umer Abbasi
03781272

Kesava Prasad
03792162

Naqash Naveed
03779712

Safi Majid
03781554

July 24, 2025

Motivation

Wildland fire-fighting now unfolds against a backdrop of record heat waves, drought, and forest loss. By 21 July 2025 the National Interagency Fire Center had logged **41,069** fires and **2.89 million acres** burned in the United States alone, which is already above the ten-year average weeks before peak season. Crews must digest satellite hotspots, UAV frames, fuel grids, wind forecasts, and hundreds of pages of SOPs while making life-critical decisions in minutes. These pressures drive five concrete requirements that shaped FireGPT.

1. **Reduce cognitive load with a single, familiar workspace.** Firefighters switch between radios, PDF viewers, GIS tools, and chat apps. A chat-centric UI with drag-and-drop documents and an always-visible map collapses those sources into one screen, freeing attention for tactics instead of tab hunting.
2. **Ground every answer in authoritative text.** Operating guidelines differ by region and change between seasons. A fixed top- k retrieval window risks hallucination or stale passages. An agentic RAG loop lets the LLM plan successive tool calls—session, regional, then global stores—until it surfaces a clause that truly governs the incident and can cite it verbatim.
3. **Inject live risk metrics, not yesterday's averages.** Wind shifts or a new spot fire can invalidate a plan in minutes. A grid-wise fire-assessment tool scores each one-kilometre cell inside a user-drawn box, feeding the numbers straight into the agent's reasoning chain so guidance is anchored to current, cell-level reality.
4. **Translate insight into immediate action.** Text alone is slow to parse under stress. The agent therefore returns structured JSON(safe drone waypoints for different operational tasks like surveillance, flanking etc) so the map layer can render flight paths without additional clicks.
5. **Survive bandwidth drops and language barriers.** All checkpoints are open weights served locally via Ollama, and embeddings plus reranking are multilingual. Crews keep full functionality when satellite links sag and can query in English, Spanish, or German without re-configuring the stack.

These five drivers, which reduce mental load, guarantee evidenced advice, track live risk, produce actionable outputs, and stay resilient in the field, form the north star for every design choice detailed in the sections that follow.

1. Project Description

Our objective was to develop a modern, web-based application—FireGPT—that enables users to submit queries, upload images and documents, and interact with a map, while meeting the defined goals and operating under the stated assumptions.

1.1. Goals

1. **Clean, Workflow-centric UI:** One of the primary goals was to deliver a modern, easy to navigate user interface that would feel instantly familiar to anyone who had used ChatGPT or any other chatbot powered by a large language model. The Inspiration was drawn from Google NotebookLM because its session based document workflow matched our need to load and reference multiple PDF files during a chat. Since spatial context is critical in fire fighting and directly affects path finding and terrain assessment, an interactive map was planned to remain visible at all times. The map would let users add geospatial inputs such as fire locations, drone positions and other points of interest, and it would display system generated routes and other location specific outputs. Figure 1 visualizes the look and feel we set out to achieve.
2. **Modular architecture:** All external capabilities—document retrieval and fire assessment—are surfaced as independent tools behind a lightweight Model Context Protocol (MCP) gateway. The language models themselves are served locally via Ollama, which presents Qwen3-8B (or any drop-in checkpoint) through the same OpenAI-compatible Chat API that the ReAct agent already understands. This double layer of abstraction—MCP for tools, OpenAI API for LLMs—cleanly decouples application logic from implementation details: you can swap Qwen3-8B for a larger model or an online endpoint, bolt on a satellite-thermal-imagery service, or migrate from ChromaDB to another vector store without touching agent code or front-end components.
3. **Evidence-grounded answers (no hallucination):** Operational guidance must cite doctrine, not “model intuition.” FireGPT therefore adopts an **agentic RAG** strategy: the LLM reasons about when and how often to call retrieval tools, climbing a three-tier ladder—session → regional → global stores—until it finds authoritative text. A multilingual embedder (bge-m3) plus cross-encoder reranker trims noise, while the agent streams back citations and a truthful, helpful, harmless response to the GUI.
4. **Vision Understanding:** Operational imagery—satellite burns scars, UAV frames, even a hand-drawn attack plan on a whiteboard—enters FireGPT through a dedicated vision-language model: Qwen2.5-VL. The checkpoint converts each image into a structured caption that lists detected objects, annotated routes, textual labels, and salient spatial relations (“fire front 200m east of access road”). Because the caption is emitted in the same JSON schema used by textual tools, the ReAct agent can fuse visual facts with retrieved SOP passages and GeoJSON POIs in a single reasoning pass. The result is a seamless workflow where users can drop a satellite tile or photo of a paper attack plan into chat and immediately receive doctrine- aligned guidance, safe waypoint suggestions, or a critique of the plan’s coverage gaps—all without preprocessing or manual transcription.
5. **Real-time situational awareness:** Users can sketch an approximate fire-region bounding box and drop POIs (e.g. “must protect hospital”). The agent grids that box, invokes the fire-

assessment tool for per-cell wind, fuel, and risk metrics, and updates advice as the geometry changes—supporting rapid, “what-if” iterations without re-embedding the entire map.

6. **Actionable output, not just text:** Beyond plain-language answers, the system returns a structured JSON with latitude–longitude waypoints for drones. The map layer renders it instantly, so operators can see a safe flight path rather than parse a paragraph under stress. Furthermore, the model is able to give fire-assessment and short-term fire spread, attack plan etc, all compliant with given SOPs.
7. **Multilingual reach, edge deployability:** All core models (Qwen, bge) are open-weights and multilingual, enabling deployment on sovereign hardware and use in mixed-language incidents. The heaviest checkpoint—Qwen3-8B—runs comfortably on a 8 GB GPU, keeping inference latency within the tight feedback loop required for field decision support. So, the whole system, runs comfortably on a 16 GB GPU.

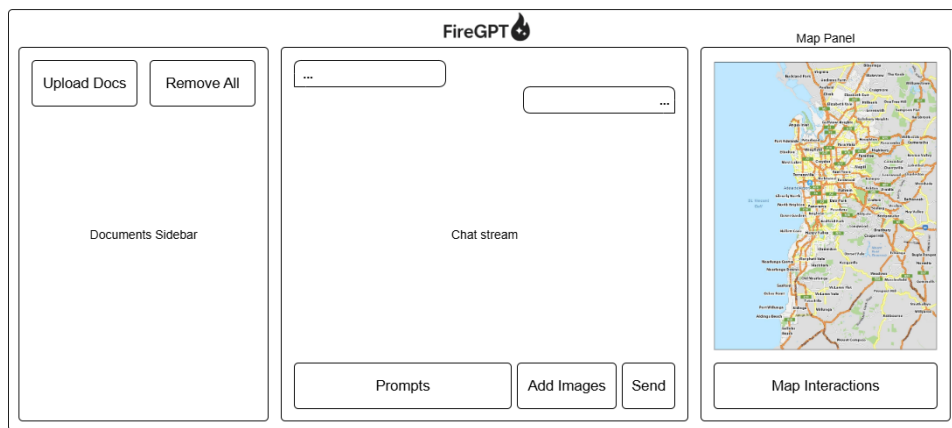


Figure 1: Conceptual user-interface mock-up showing the desired three-pane layout

1.2. Assumptions

The following premises frame the scope, deployment model, and evaluation of FireGPT. They are not hard requirements for future versions, but they shape the design and should be kept in mind when interpreting results.

1. **Field hardware.** A single workstation or edge server equipped with at least a 16 GB VRAM GPU is available at the incident command post. Qwen3-8B and Qwen2.5-VL can run on CPU fall-back, but latency targets assume GPU inference.
2. **Internet connectivity.** The system can operate fully offline with real-time fire assessment. However, with an internet connection, FireGPT would produce much better recommendations.
3. **Legal access to documents.** All SOPs, after-action reports, and manuals uploaded to the session store are assumed to be cleared for operational use and compliant with agency copyright or classification rules.

4. **Language coverage.** Embeddings and reranking are multilingual, yet systematic evaluation has been performed on English and German corpora only. Answers in other languages may exhibit degraded style or formulaic phrasing, though semantic accuracy is expected to hold.
 5. **Image characteristics.** Vision captioning assumes JPEG or PNG inputs.
 6. **User expertise.** Operators have basic familiarity with incident-command workflow and map-based annotation but require no prior knowledge of prompt engineering or LLM tool syntax.
-

2. Data

This section details the dataset compiled for the FireGPT. Data collection was performed in three focused phases: **Regional**, **Global**, and **Session Documents**. Each phase contributes uniquely to building a robust, realistic, and diverse training base for a retrieval-augmented generation (RAG) pipeline, intended for firefighting strategy generation.

All documents were acquired through extensive manual searching via Google, verified against authoritative sources such as official government portals, fire coordination groups, and inter-agency safety organizations. Only publicly accessible or open-access documents were selected, and preference was given to PDF formats due to their ease of parsing and standardized formatting.

We validated our selections using 20 domain-specific prompts designed to simulate real fire situation queries. These prompts allowed us to test system responses and refine the embedded knowledge, ensuring coverage of varying terrain, fire types, coordination complexity, and national firefighting doctrines.

2.1. Regional Documents

This subset includes authoritative operational manuals, strategy documents, and incident reports from U.S. firefighting agencies. The U.S. has the most mature and openly documented aerial firefighting ecosystem, making it suitable for baseline training.

1. **MAFFS Operating Plan (2025)** — *U.S. Air National Guard* [1]
Outlines coordinated use of military aircraft in civil firefighting; essential for civil-military response modeling.
2. **Standards for Airtanker Operations (2022)** — *USDA Forest Service* [2]
Contains tactical safety and coordination practices; critical for understanding fixed-wing deployment logic.
3. **Federal Aerial Firefighting BRP Report (2002)** — *USDA/USFS* [3]
Offers accident investigation and reform recommendations; foundational for strategic risk modeling.
4. **Woolsey Fire AAR (2018)** — *LA County Fire Department* [4]
A case study in complex multi-agency response; supports operational realism testing.
5. **California 2016 Fire Season AAR** — *CAL FIRE* [5]
Captures lessons learned from multiple wildland fire events; used to analyze strategy under varying conditions.

2.2. Global Documents

These documents come from diverse international sources and expand the system's geographical and environmental understanding. Including multiple ecosystems (Nordic, Mediterranean, Alpine, Australian bushland) helps ensure model generalizability and robustness.

1. **NAFC Aerial Strategy (2021–2026)** — *Australia, National Aerial Firefighting Centre* [6]
National plan; shows large-scale structured doctrine and funding allocation logic.

2. **ATSB Safety Review (2000–2020)** — *Australia, ATSB* [7]
Longitudinal safety review; critical for identifying systemic patterns in air operations.
3. **Aerial Effectiveness Review – Part 1** — *Australia, CSIRO* [8]
Empirical data on suppression outcomes; useful for scenario ranking logic.
4. **Aerial Suppression Literature Review (2021)** — *Australia, CSIRO* [9]
Summarizes global aerial tactics; valuable for knowledge base generalization.
5. **MSB Skogsbrandsläckning Guide (2022)** — *Sweden, MSB* [10]
Swedish national tactics in forest suppression; introduces Nordic climate considerations.
6. **EFFIS Annual Report (2023)** — *Europe, JRC* [11]
Cross-border statistics, climate impacts, and geospatial aerial data.
7. **CIFFC Aviation Workshop Report (2023)** — *Canada, CIFFC* [12]
Covers Canadian doctrine and workshop insights; enhances multilateral response simulation.

2.3. Session Documents

These documents were not directly used in training but were reserved for testing and evaluation. They were intentionally selected for their structure, depth, and instructional value to validate RAG responses.

1. **NWCG Standards for Aerial Supervision (PMS 505)** — *NWCG, USA* [13]
Clearly structured guidance on airborne command; ideal for evaluating retrieval accuracy.
2. **NWCG Airtanker Base Operations (PMS 508)** — *NWCG, USA* [14]
Focuses on logistics and resource dispatch; useful for testing planning module accuracy.
3. **NWCG Helicopter Operations (PMS 510)** — *NWCG, USA* [15]
Operational detail on rotorcraft use in varied terrain; adds tactical diversity for test cases.
4. **DFV Aerial Firefighting Recommendation (2022)** — *DFV, Germany* [16]
Germany's only formal aerial coordination guide; essential for modeling EU-aligned air-ground protocol.

2.4. Data used in Fire Assessment

1. **Global Forecast System (GFS)**: Fetches `u_component_of_wind_10m_above_ground`, `v_component_of_wind_10m_above_ground`, and `temperature_2m_above_ground` for the specified `forecast_hours`. It includes robust error handling to attempt a broader time range if no data is found for the exact forecast time.
2. **EC JRC Global Forest Cover 2020, V2**: Provides binary forest/non-forest presence at 10m resolution.
3. **UMD/Hansen Global Forest Change 2024**: Provides `treecover2000` (percentage tree cover in 2000) and `lossyear` (year of forest loss).
4. **Copernicus CORINE Land Cover (CLC) 2018**: Offers land cover classification at 100m resolution, crucial for identifying fuel types.

5. **SRTM Digital Elevation Model (DEM):** Provides altitude data.
6. **Overpass API:** Provides point of interest data.

2.5. Preprocessing

Global and region-level documents were ingested beforehand, are bundled with the software and enable semantic retrieval. Session-level documents are also pre-processed and ingested during the session in the same way. Both sets undergo identical processing steps, outlined below.

1. **Multilingual Handling:** A multilingual, embedding model BAAI/bge-m3, was used to process both English and German documents.
 2. **Chunking:** Text is split into 1024-token segments, despite a 32k context window, to sharpen retrieval precision, since smaller, idea-focused vectors rank more cleanly than sprawling blocks while still allowing hundreds of segments to be returned at once. A 128-token overlap is maintained so consecutive chunks share enough state for chain-of-thought continuity and information that straddles boundaries remains discoverable, boosting recall with minimal storage overhead. Paragraph and newline breaks are respected, ensuring that semantic units stay intact and are reasoned over coherently by the agentic RAG system. Embeddings are produced by the local BAAI/bge-m3 model and stored in Chroma for low-latency similarity search.
 3. **Image Handling:** All PDF documents were text-extracted only. Image ingestion and semantic retrieval is earmarked as a potential extension in future work.
-

3. System Description

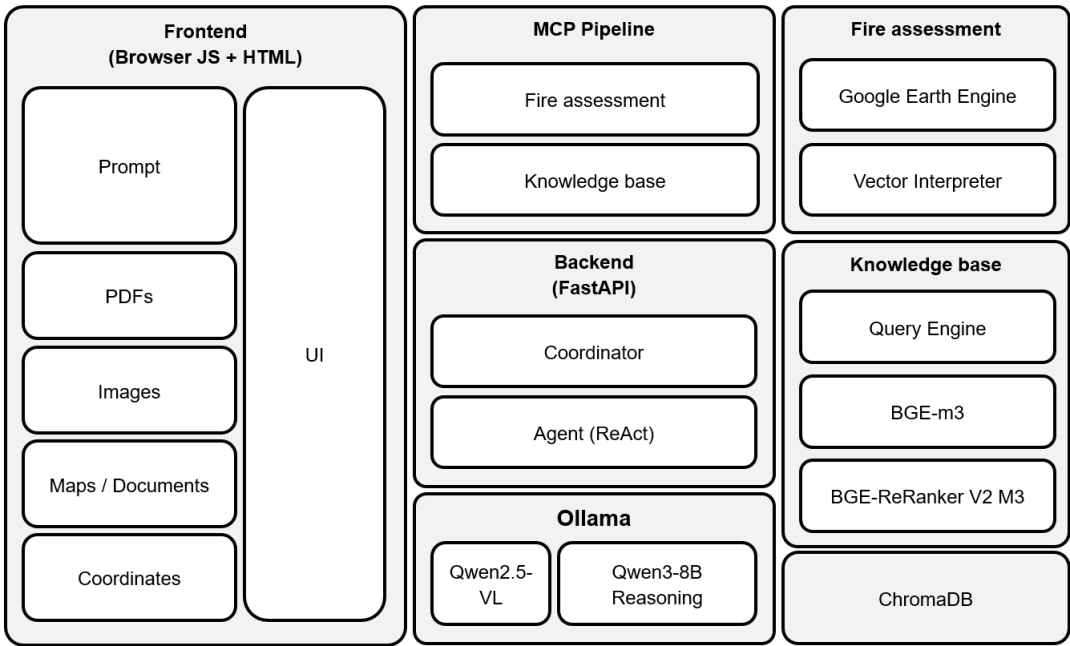


Figure 2: High-level system overview of FireGPT, illustrating major blocks.

FireGPT has 4 major blocks, as illustrated in fig. 2.

1. **Multimodal front-end.** As illustrated in fig. 3, users can submit SOPs, after-action reports, or other documents (PDF), aerial or ground imagery, a map-based fire perimeter, points of interest, and free-form natural-language questions in one unified interface.
2. **AI back-end.** A Fast-API backend, running vision-language ReAct agent processes the entire input bundle and streams back an answer in plain language. When appropriate, it also emits a structured JSON payload of latitude–longitude waypoints for downstream visualisation or autonomous drone control. The agent connects to the VLM/LLM via Ollama and accesses both the knowledge base and FireAssessment services over a Model Context Protocol (MCP) server.
3. **Knowledge base.** To ensure its responses are grounded in authoritative sources, the agent executes a three-stage retrieval cascade: it searches the user’s own uploads first, then a region-level knowledge base, and finally a global repository.
4. **Fire assessment.** Throughout this workflow it can also invoke a fire-risk assessment tool scores terrain cells forest area type, wind, weather, historical forest loss, and proximity to critical infrastructure.

From the user’s point of view, this means they can request anything from context-specific SOP excerpts and attack-plans to automatically generated drone flight paths and operating instructions, all without leaving the FireGPT interface.

Although every component (document-ingestion pipeline, retrieval stack, and vision–language model) is multilingual by design, our systematic evaluation has so far focused on English material, with spot checks on German documents yielding comparable results.

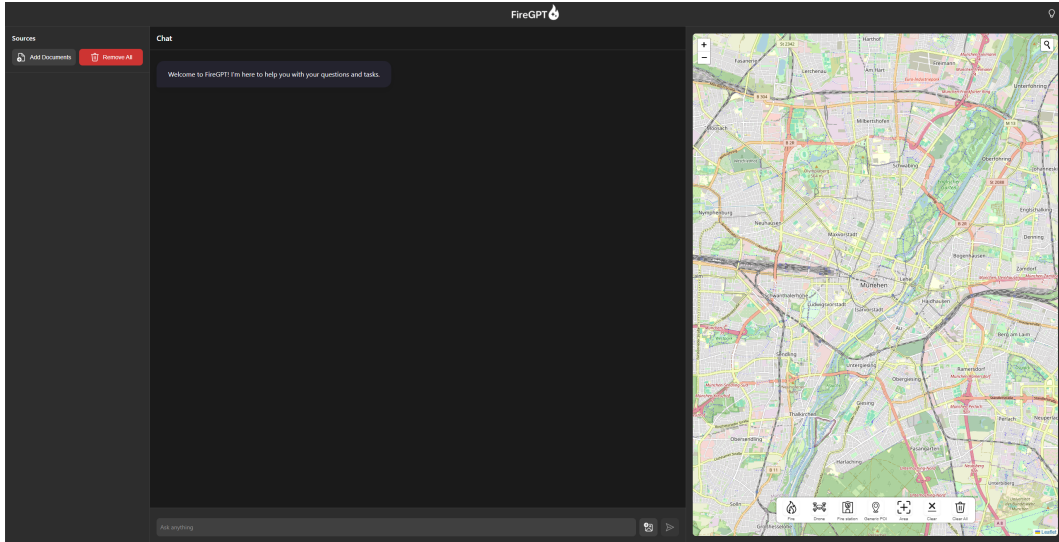


Figure 3: FireGPT front-end: users can upload documents, upload images, draw a bounding box to provide geospatial context for a wildfire query, and mark different points of interests.

4. Architecture

4.1. Multimodal Front-end

The graphical interface shown in fig. 3 is built with stand-alone JavaScript, HTML5, and CSS3. It presents three coordinated input channels:

1. **Document drop-zone.** Users may drag-and-drop additional PDFs (SOPs, incident reports, manufacturer manuals, etc.) at any point in the dialogue, allowing the system to ingest new evidence on the fly.
2. **Image uploader.** The same panel accepts satellite tiles, drone photographs, or hand-drawn sketches such as annotated attack plans, widening the visual context available to the VLM agent.
3. **Interactive map canvas.** Powered by Leaflet and OpenStreetMap, the canvas lets users draw polygons or bounding boxes to delineate a fire perimeter, drop markers for drone take-off/landing and refill points, flag critical infrastructure (e.g. hospitals), and attach free-text notes (“must protect”). All geometry is exported as GeoJSON and passed to the back-end unchanged, preserving spatial fidelity.

A chat bar anchors the layout, so natural-language queries, uploaded documents, images, and geospatial annotations flow into FireGPT as a single multimodal bundle. The guiding design principle is minimal cognitive overhead: users interact with familiar map gestures, standard file uploads,

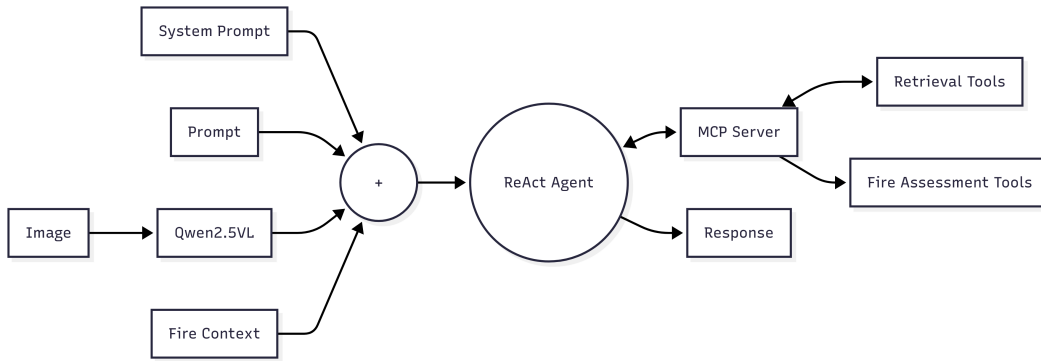


Figure 4: High-level back-end flow. A vision–language captioner (Qwen2.5-VL) turns the incoming image into text, which is concatenated with the system prompt, user prompt, and geospatial fire context before being handed to the ReAct agent. The agent may call retrieval and fire-assessment tools via a Model Context Protocol (MCP) server, then streams the final response back to the front-end.

and plain text, while the system quietly harmonises these modalities for the downstream Reason and Action (ReAct) agent.

4.2. AI Back-end

Backend Overview. Our design draws on two external references: A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges [17] and OpenAI’s GPT-4.1 Prompting Guide [18]. Building on the guidelines distilled in these works, FireGPT integrates the following components in its pipeline, as illustrated in fig. 4:

1. **Vision captioning.** Each uploaded image is first passed through Qwen2.5-VL to produce a rich, structured caption.
2. **Message assembly.** The system prompt, user query, image caption, and GeoJSON fire context are packed into a LangGraph message list.
3. **ReAct reasoning.** The bundle is forwarded to a ReAct agent (built with LangGraph) whose principal LLM is Qwen3-8B running on an Ollama server.
4. **Tool invocation.** Guided by the ReAct loop, the agent can access tools on an MCP server and dynamically decides whether to:
 - retrieve documents (session → regional → global), or
 - call the fire-assessment tool using the provided fire context for per-cell risk metrics.
5. **Response synthesis.** The agent merges tool outputs with its own chain-of-thought and streams back a plain-language answer, optionally accompanied by JSON waypoints.
6. **Response Parsing.** As we are using a reasoning model, agent’s responses includes it’s own thoughts as well. So, we strip off the the thoughts (anything inside <think> ... <\think> tags), parse the waypoints json, and send back clean response and waypoints json to the GUI.

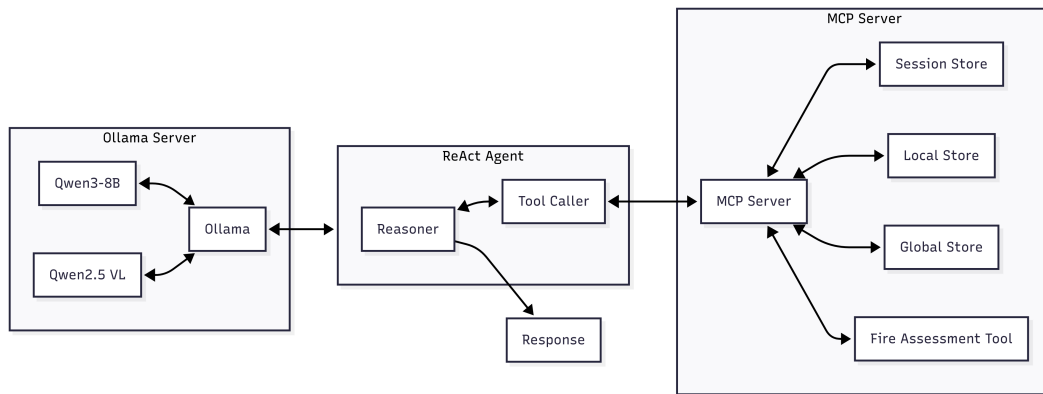


Figure 5: Component view. The Ollama server hosts the two Qwen models. The ReAct agent contains a Reasoner (LLM) and a Tool Caller. All external tools live behind the MCP server: three ChromaDB stores (session, regional, global) and a fire-assessment micro-service.

Zoom into Major Subsystems. Figure 5 zooms in on the three collaborating major subsystems. On the left, an Ollama inference node hosts both Qwen3-8B (text) and Qwen2.5-VL (vision) behind a single, low-latency HTTP endpoint. In the centre, the ReAct agent splits into a Reasoner, which plans tool invocations, and a Tool-Caller, which executes the plan and feeds results back for further reasoning. On the right, a Model Context Protocol (MCP) server exposes four stateless micro-services through one streaming API: a session-specific vector store, a regional knowledge base, a global knowledge base, and a fire-assessment engine. Together these components give the agent immediate access to both authoritative documents and real-time risk analytics while keeping the overall latency close to that of a single LLM call.

System-prompt Optimisation Workflow. Figure 6 visualises the human-in-the-loop procedure we followed to harden the FireGPT system prompt. We began with an initial template and a hand-labelled set of twenty representative queries paired with “gold” answers. The labelled set was stratified into train, dev, and test splits so that prompt edits could be scored on held-out data. Using the training portion, we iteratively rewrote the template by adjusting role instructions, tool-call format cues, and safety constraints, until the dev split reached a satisfactory success rate. Finally, we estimated the prompt’s true performance on the untouched test split.

4.3. Knowledge Base

We built a hierarchical knowledge base, comprising of region-level document, global-level documents and session level documents. Global and regional corpora are pre-embedded with the BAAI/bge-m3 model, enabling multilingual search out of the box. FireGPT ships with those two corpora and a session store is created on-the-fly as users upload new PDFs.

Figure 7 highlights the retrieval stack:

- **Stage 1 – Dense retrieval.** User queries are embedded with bge-m3 and compared against vector stores in ChromaDB, producing 20 candidate chunks.

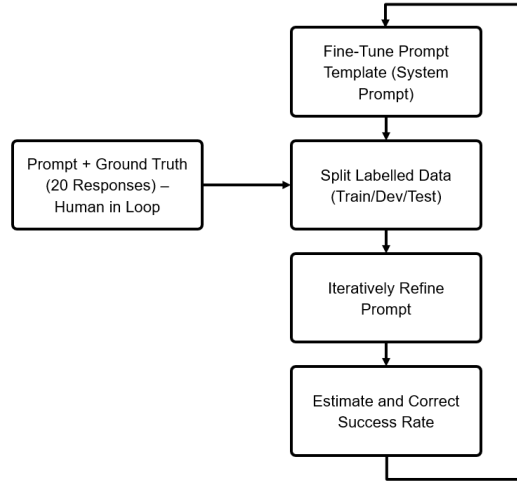


Figure 6: Flowchart showing the optimization of scheme of system prompt.



Figure 7: Two-stage retrieval pipeline. An embedding search over ChromaDB fetches the top-20 candidates, which are reranked by the reranker, returning the five highest-scoring chunks to the agent.

- **Stage 2 – Cross-encoder reranking.** The candidates are re-scored by `bge-reranker-v2-m3` and the top five are supplied to the ReAct agent.

This pipeline keeps latency low while maintaining semantic accuracy across languages. Although formal evaluation has concentrated on English, but the spot checks on German showed comparable performance.

4.4. Fire Assessment

Fire Assessment tool can be called with the bounding box of fire perimeter. A general overview is depicted in Figure 8. It queries an API which provides near real-time fire danger assessments for specified geographical areas by leveraging geospatial data from Google Earth Engine (GEE) and Point of Interest (POI) data from OpenStreetMap (OSM).

The system's core functionality is exposed through the `/assess-fire-danger` endpoint. This endpoint accepts a geographical bounding box and other parameters, and returns a detailed assessment. The process involves:

1. **Subgrid Processing:** The specified area is divided into smaller subgrids.
2. **Data Fetching:** For each subgrid, the system fetches various environmental and meteorological data.

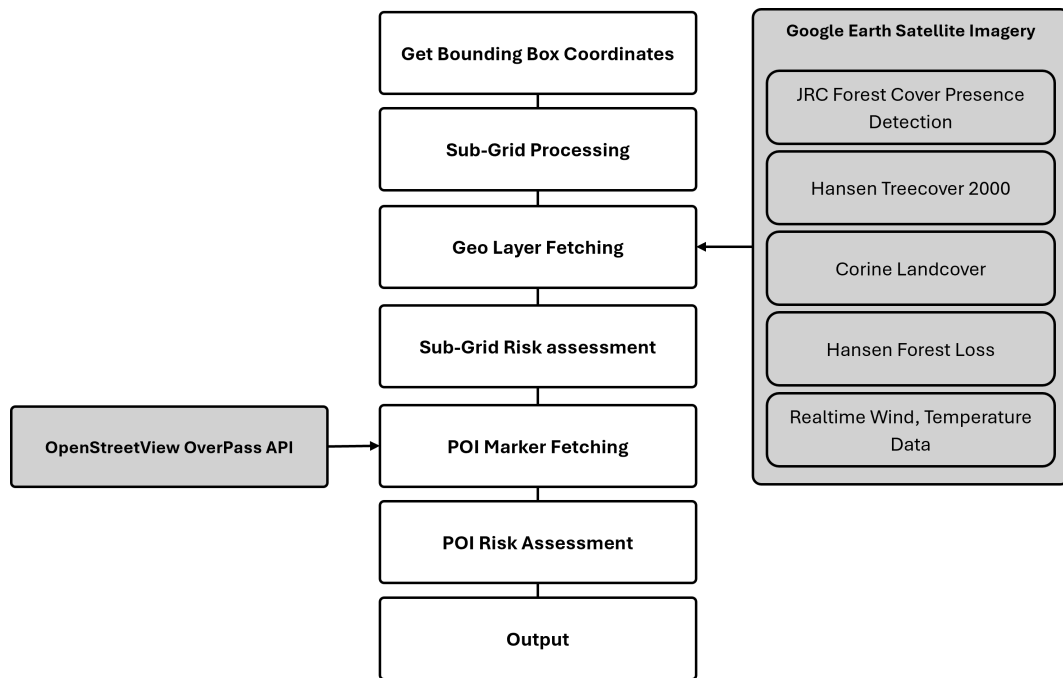


Figure 8: Workflow of the fire assessment engine: Marked gray parts are external internet APIs. The process moves from Top-Bottom with Sideways on applicable processes.

logical parameters from GEE, including forest cover, land cover, wind speed, temperature, and elevation.

3. **Danger Calculation:** A rule-based model calculates a fire danger score for each subgrid based on the fetched data. Factors like fuel availability, weather conditions, and topography are considered.
4. **POI Identification:** The system fetches relevant Points of Interest (e.g., hospitals, fire stations) from OpenStreetMap and determines their proximity to high-danger areas.

The final output is a comprehensive JSON response containing the danger assessment for each subgrid and a list of key POIs with their relationship to the danger zones. This provides actionable intelligence for emergency response and public safety. The entire system is containerized using Docker for portability and scalability.

4.5. Prompt-to-Response Workflow

Figure 4 illustrates the life-cycle of a single user request:

1. **Input bundling.** GUI uploads a document which is ingested into the session store asynchronously. Then, GUI transmits a single JSON payload that contains images, the natural-language query, and/or geospatial context (fire perimeter and POI markers in GeoJSON).

2. **Vision captioning.** Each image is routed through Qwen2.5-VL (served via Ollama) to obtain a structured and detailed textual caption.
3. **Message assembly.** The system prompt, user query, image caption(s), and GeoJSON are concatenated into a LangGraph message list and forwarded to the ReAct agent.
4. **Tool-aware reasoning.** Powered by Qwen3-8B, the agent enters the ReAct loop and decides in real time whether to
 - retrieve documents—first from the session store, then the regional store, and finally the global store, and/or
 - call the fire-assessment service using fire perimeter for per-cell risk, weather, fuel, and proximity metrics.

All tool calls are executed through the MCP server and streamed back to the agent for further reasoning.

5. **Response synthesis.** The agent fuses tool outputs with its own chain-of-thought and drafts a reply that may include a JSON block of latitude–longitude waypoints (e.g., drone flight paths).
6. **Post-processing and rendering.** Before the answer reaches the front-end, any internal `<think>...</think>` segments are stripped, the waypoint JSON is parsed, and the GUI receives a clean textual response plus map-ready overlays that are immediately plotted on the interactive canvas.

5. Model Suite and Rationale

FireGPT assembles four open-weights models, each chosen to excel at a distinct link in the chain from raw field data to grounded advice: reasoning and tool-use planning, vision captioning to textualise imagery for the agent, multilingual dense retrieval across heterogeneous document sources, and precision reranking so the agent reasons over the right text.

Qwen3-8B (primary reasoning & tool planner). We selected Qwen3-8B as the agent’s core LLM for three reasons. First, headroom: its larger sibling (30B) tops published reasoning, math, and instruction-following benchmarks (see fig. 9), and that capability transfers well to smaller sizes in the same family (empirically, we observed similar tool selection patterns across 30B→8B down-scaling). Second, controllability: Qwen3 uniquely exposes a `think` / `fast` execution toggle, allowing FireGPT to dial up structured chain-of-thought when the agent must plan multi-step tool calls (e.g., `retrieve`→`assess`→`summarise`), and dial it down for rapid chat-style follow-ups. Third, robustness in practice: in our internal dry-runs the model generated well-formed JSON tool arguments far more reliably than `mistral-7b-instruct-v0.3`, which frequently omitted required fields. Together these factors made Qwen3-8B the best balance of reasoning power, tool-API reliability, permissive licensing, and 8 GB-class hardware footprint. Furthermore, a 4-bit quantized checkpoint was used to achieve a balance between speed and accuracy.

	Qwen3-30B-A3B MoE	QwQ-32B	Qwen3-4B Dense	Qwen2.5-72B-Instruct	Gemma3-27B-IT	DeepSeek-V3	GPT-4o 2024-11-20
ArenaHard	91.0	89.5	76.6	81.2	86.8	85.5	85.3
AIME'24	80.4	79.5	73.8	18.9	32.6	39.2	11.1
AIME'25	70.9	69.5	65.6	15.0	24.0	28.8	7.6
LiveCodeBench v3, 2024.10-2023.102	62.6	62.7	54.2	30.7	26.9	33.1	32.7
CodeForces Elo Rating	1974	1982	1671	859	1063	1134	864
GPQA	65.8	65.6	55.9	49.0	42.4	59.1	46.0
LiveBench 2024-11-25	74.3	72.0	63.6	51.4	49.2	60.5	52.2
BFCL v3	69.1	66.4	65.9	63.4	59.1	57.6	72.5
MultiIF 8 Languages	72.2	68.3	66.3	65.3	69.8	55.6	65.6

1. AIME'24/25: We sample 64 times for each query and report the average of the accuracy. AIME'25 consists of Part I and Part II, with a total of 30 questions.
2. Alder: We didn't activate the think mode of Qwen3 to balance efficiency and effectiveness.
3. BFCL: The Qwen3 models are evaluated using the FC format, while the baseline models are assessed using the highest scores obtained from either the FC or prompt formats.

Figure 9: Representative benchmark results published by Alibaba Cloud. Larger Qwen3 checkpoints (30B illustrated) consistently lead on multi-task reasoning suites versus Gemma, DeepSeek, and closed models such as GPT-4o in the reported comparisons. We deploy the 8B variant for a latency/VRAM trade-off while retaining strong reasoning behaviour.

<https://qwenlm.github.io/blog/qwen3/>

Qwen2.5-VL (vision captioner). We chose Qwen2.5vl (7B) as our vision captioner because its larger siblings outperforms other state-of-the-art models across a number of benchmarks (fig. 10). While Qwen2.5-VL is a capable multimodal LLM, fig. 9 shows that text reasoning in the newer Qwen3 line outperforms it by a wide margin (30B vs. 72B illustrated). We therefore adopt a specialisation strategy: Qwen2.5-VL (7B checkpoint) is used solely to convert uploaded satellite/drone imagery and user plans/sketches into rich captions that the Qwen3-8B reasoner can ingest as text. This split keeps caption latency low and concentrates scarce GPU memory where sophisticated tool reasoning is needed most. (Once Qwen3-VL is publicly available, we plan to collapse to a single multimodal checkpoint.) Furthermore, a 4-bit quantized checkpoint was used to achieve a balance between speed and accuracy.

BAAI/bge-m3 (multilingual embeddings). Wildfire operations span jurisdictions; SOPs, situation reports, and equipment manuals appear in multiple languages (English, Spanish, German, ...). The bge-m3 embedding model offers strong cross-lingual semantic alignment (fig. 11), letting a single query retrieve relevant passages regardless of source language. Its 768-dimensional output keeps ChromaDB storage compact and accelerates approximate nearest-neighbour search, which matters when session uploads grow mid-incident.

BAAI/bge-reranker-v2-m3 (cross-encoder reranking). Dense retrieval alone returns a coarse candidate set; noisy or partially translated documents can surface near the top. We therefore

Dataset	Qwen2.5-VL-3B (📹📄)	Qwen2.5-VL-7B (📹📄)	Qwen2.5-VL-32B (📹📄)	Qwen2.5-VL-72B (📹📄)	Gemini-2 Flash	GPT-4o	Claude3.5 Sonnet
MMMU	53.1	58.6	70.0	70.2	70.7	70.3	70.4
MMMU Pro	31.6	38.3	49.5	51.1	57	54.5	54.7
DocVQA	93.9	95.7	94.8	96.4	92.1	91.1	95.2
InfoVQA	77.1	82.6	83.4	87.3	77.8	80.7	74.3
CC-OCR	74.5	77.8	77.1	79.8	73.0	66.6	62.7
OCRBenchV2	54.3/52.1	56.3/57.2	57.2/59.1	61.5/63.7	-	46.5/32.3	45.2/39.6
MegaBench	28.9	36.8	-	51.3	55.2	54.2	52.1
MMStar	55.8	63.9	69.5	70.8	69.4	64.7	65.1
MMBench1.1	81.5	84.3	84.6	88.0	83.0	82.1	83.4
MathVista	62.3	68.2	74.7	74.8	73.1	63.8	65.4
MathVision	21.2	25.1	38.4	38.1	41.3	30.4	38.3
VideoMME	61.5/67.6	65.1/71.6	70.5/77.9	73.3/79.1	-/-	71.9/77.2	60/62.9
MMBench-Video	1.63	1.79	1.93	2.02	-	1.68	1.38
LVBench	43.3	45.3	49.0	47.3	-	30.8	-

Figure 10: Representative benchmark results published by Alibaba Cloud. Larger Qwen2.5vl checkpoints (32 and 72B illustrated) consistently outperforming GPT-4o and matching or exceeding Gemini-2 Flash across multimodal, document-QA, reasoning, math, and video benchmarks.

<https://github.com/QwenLM/Qwen2.5-VL>

rerank the top- k (default $k = 20$) candidates with the multilingual bge-reranker-v2-m3 cross-encoder, which materially improves early precision (fig. 12). Higher-quality top-5 evidence reduces “hallucinated SOP” failure modes and shortens the ReAct loop, because the agent needs fewer follow-on retrieval calls to answer.

6. Tools and Rationale

FireGPT treats external capabilities—document search and fire-risk analytics—as tools served over a lightweight Model Context Protocol (MCP). All tool calls are planned by the ReAct agent in real time, an **agentic RAG** style that differs from classic RAG pipelines where context is pre-assembled by deterministic rules.

Why agentic RAG?

1. **Dynamic reasoning.** Wildfire queries vary wildly (“show me SOP section 7”, “optimise this drone route”). A static top- k context window either over-retains irrelevant text or misses specialised material. Letting the agent decide when, how often, and with which query to retrieve (i.e., if retrieved docs are not good enough, agent’s reasoner can optimize the query can retrieve again) yields higher factual F_1 .
2. **Tool composability.** The agent can chain tools—e.g., retrieve → fire-assess → retrieve

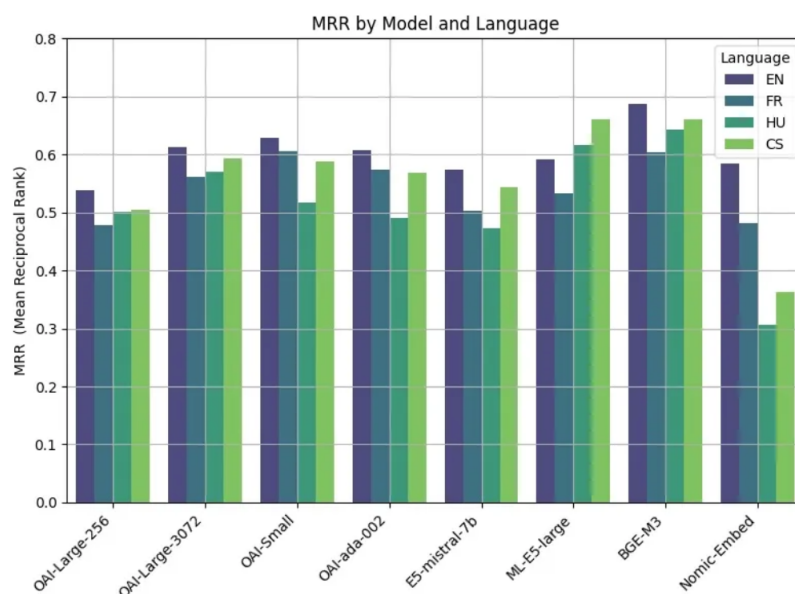


Figure 11: Embedding model comparison (selected multilingual MTEB-style tasks). `bge-m3` leads or matches state-of-the-art multilingual encoders while producing compact 768-d vectors, making it well-suited for large mixed-language corpora.

<https://huggingface.co/BAAI/bge-m3>

again—without hand-crafted orchestration, a pattern difficult to express in traditional RAG pipelines.

Why three document stores (session, regional, global)?

1. **Session store.** Captures user-uploaded PDFs mid-incident; freshness is paramount, so embeddings live in-memory for sub-second updates.
2. **Regional store.** Holds pre-ingested SOPs and historical reports for the jurisdiction in which the fire occurs. Keeps answers grounded in locally approved doctrine (e.g., air-tanker altitude limits differ between Canada and Spain).
3. **Global store.** A long-tail fallback for rare queries (“lithium-battery warehouse fire”) where regional material is thin. The ReAct agent consults this tier only when previous searches return low relevance scores, limiting noise.

This tiered design mirrors the cognitive hierarchy of an experienced fire-officer: start with the freshest incident data, expand to local practice, and finally consult global references only if needed. Combined with the agentic loop, FireGPT answers remain both contextual and evidence-backed, without overwhelming the user or the GPU with unfiltered text. Figure 7 (p. 13) illustrates the retrieval and reranking flow that underpins these stores.

embedding model	bge-en-v1.5 large		bge-m3		openai-small		openai-large		mxbai-embed-large-v1	
reranker	mrr	hit rate	mrr	hit rate	mrr	hit rate	mrr	hit rate	mrr	hit rate
without reranker	65.07	85.1	69.67	88.94	65.69	89.42	67.37	90.38	66.66	88.46
bge-reranker-base	75.77	90.87	77.48	94.23	75.75	93.27	76.3	94.23	76.63	91.83
bge-reranker-large	75.86	90.87	78.66	94.23	77.09	94.23	77.08	95.67	77.24	92.31
mxbai-rerank-large-v1	72.77	88.46	75.99	93.27	74.62	91.35	74.32	92.31	73.89	89.9
jina-reranker-v1-base-en	75.81	89.9	79.44	93.75	77.64	91.83	77.85	92.79	76.96	91.83
cohere rerank	75.17	90.38	76.23	91.35	76.98	92.79	76.68	93.27	76.43	92.31
ms-marco-MiniLM-L-6-v2	67.92	86.54	69.83	90.38	69.2	88.46	67.99	90.38	68.57	87.98
bge-reranker-v2-m3	78.26	90.87	80.76	94.71	79.38	93.27	79.7	94.71	79.1	92.31
bge-reranker-v2-gemma	75.19	89.9	78.14	93.75	76.74	92.31	76.28	92.31	77.25	91.83
bge-reranker-v2-minicpm-20	81.31	91.83	83.77	95.67	81.92	94.71	83.43	95.19	82.11	92.79
bge-reranker-v2-minicpm-28	81.93	91.83	84.74	95.67	84.01	94.71	83.93	95.19	82.99	93.27
bge-reranker-v2-minicpm-40	80.89	91.83	83.29	95.67	82.89	94.71	82.33	95.19	81.45	92.79

Figure 12: Retrieval ablation. Adding the multilingual bge-reranker-v2-m3 cross-encoder boosts both hit rate and MRR over pure dense retrieval.

<https://huggingface.co/BAAI/bge-reranker-v2-m3>

Why capture geospatial annotations? Wildfire responders rarely know the exact perimeter at the moment they open FireGPT, yet they do know where to start looking. Allowing the user to draw a fire-region bounding box serves as an "uncertainty buffer"—the agent can focus computation inside that window without risking an empty query or paying to tile an entire province. Likewise, free-form **points of interest (POIs)** let crews encode local tac-intel that never appears in doctrine: emergent spot fires, preferred drone launch pads, refill ponds, fire-stations, or critical infrastructure such as hospitals. Each marker carries an optional note (e.g., "must protect"), giving the LLM a structured hook for prioritisation.

Why use Fire Assessment and how the agent uses the GeoJSON? All user-drawn geometry is forwarded unmodified to the ReAct agent. Internally the agent:

- feeds the bounding box to the `fire-assessment` tool, which returns per-cell risk scores, wind vectors, fuel models, and recent forest-loss data;
- overlays user POIs onto the same grid so each cell knows what it contains (fire front, drone pad, hospital, ...);
- plans tool calls accordingly:
 - retrieval—pull SOP passages relevant to high-risk cell types (steep slopes, lithium storage);
 - reasoning—rank candidate drone waypoints that avoid high-flame-length cells and honour "must protect" POIs.

Because the assessment is grid-wise, the agent can iterate at human timescales: adjust the box, add a new POI, re-plan—no need to rerun a full-scene simulation. The net effect is advice and flight paths that are operationally safe (avoid dangerous cells), doctrine-aligned (cite the correct SOP snippets), and task-relevant (respect user annotations), closing the loop between field knowledge and AI reasoning.

Why MCP as the transport layer?

1. **Uniform I/O.** Every tool—retrieval or analytics—exposes the same JSON-in/JSON-out stream API. The agent code therefore never changes when a tool is swapped or versioned.
2. **Language-agnostic services.** MCP messages travel over plain HTTP/2; tools can be written in Python, Rust, or even serverless WASM without touching the core agent.
3. **Back-pressure & observability.** The protocol carries token-level partial results and timing metadata, simplifying timeout handling and real-time UX streaming.

Why containerization with Docker? To ensure portability and simplify deployment, the entire FireGPT backend is containerised using Docker and orchestrated with Docker Compose. This setup modularises each component of the system into its own service, promoting a clean and maintainable architecture. The key services include:

1. **ami-backend:** A FastAPI service that hosts the main application logic, built from a base Conda environment to manage Python dependencies. It communicates with both the MCP and Ollama services.
2. **ami-mcp:** The Multi-Channel Proxy service, also based on the Conda environment, which handles specific model interactions and exposes its services.
3. **ollama:** An Ollama service that runs the large language models, configured to use GPU resources for optimal performance.

This multi-container application is defined in a `docker-compose.yml` file, which manages service discovery, networking, and volume mounts for models and data. Large Hugging Face models are managed outside the Docker images via volume mounts, reducing image size and allowing for easier model updates without rebuilding images. This containerised approach provides a robust and scalable environment for deploying the Forest Fire Danger Assessment API and its dependencies.

7. Results and Discussion

FireGPT was evaluated on a purpose-built test dataset designed to reflect three mission-critical firefighting tasks: drafting attack plans from standard operating procedures (SOPs), critiquing attack plans from ground-truth images, and computing optimal drone waypoint routes. For each task, five prompt-and-reference pairs (“gold responses”) were manually authored, yielding 15 test cases in total. Model outputs were then assessed by human raters against the corresponding gold responses, allowing us to judge FireGPT’s multi-modal reasoning, agentic RAG capabilities, and real-time situational-awareness performance.

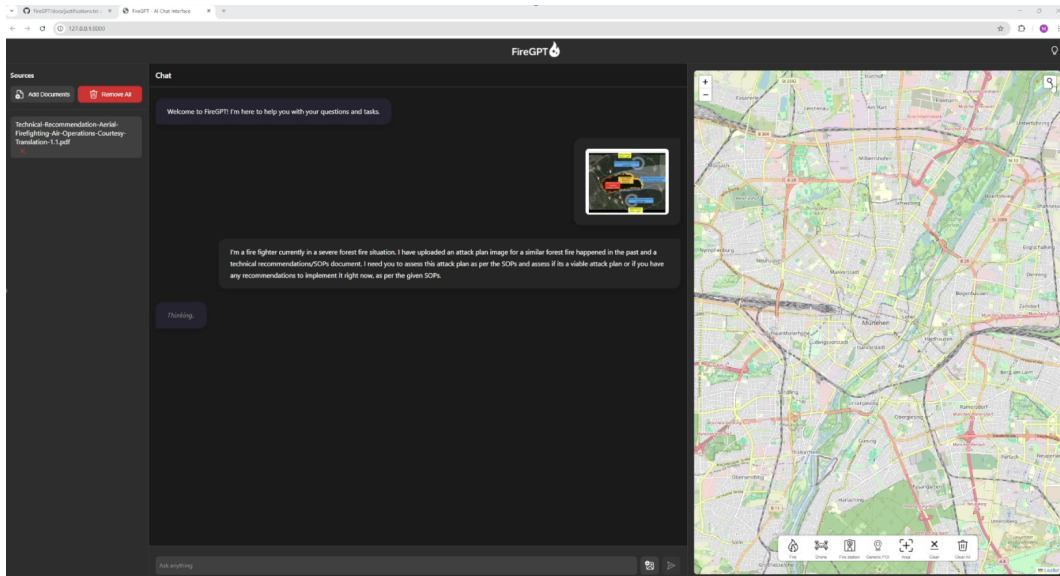


Figure 13: UI Thinking for response to create attack plan based on SOPs and Images provided in the prompt

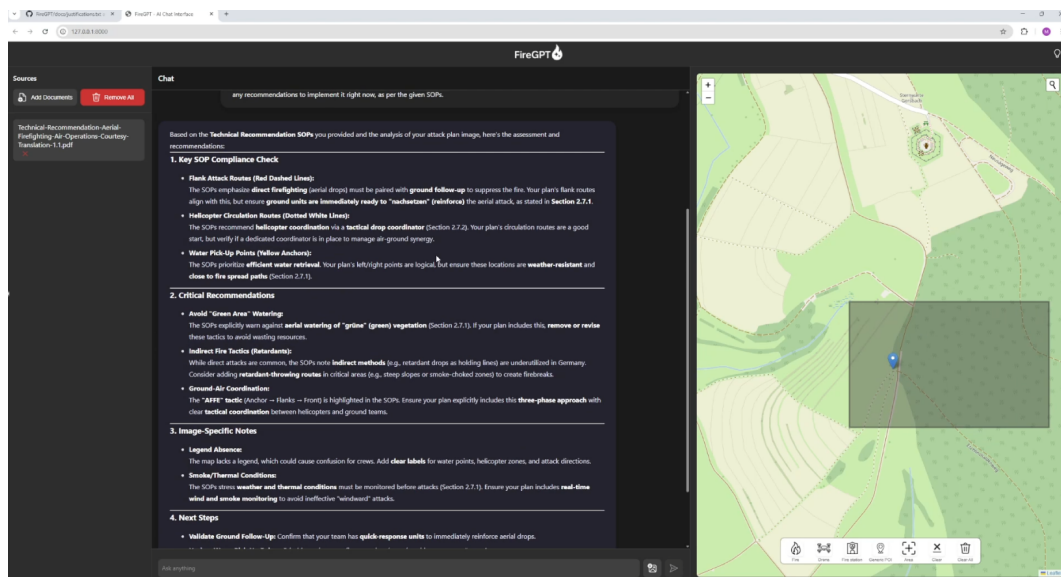


Figure 14: Fully generated prompt response from prompt in fig. 13

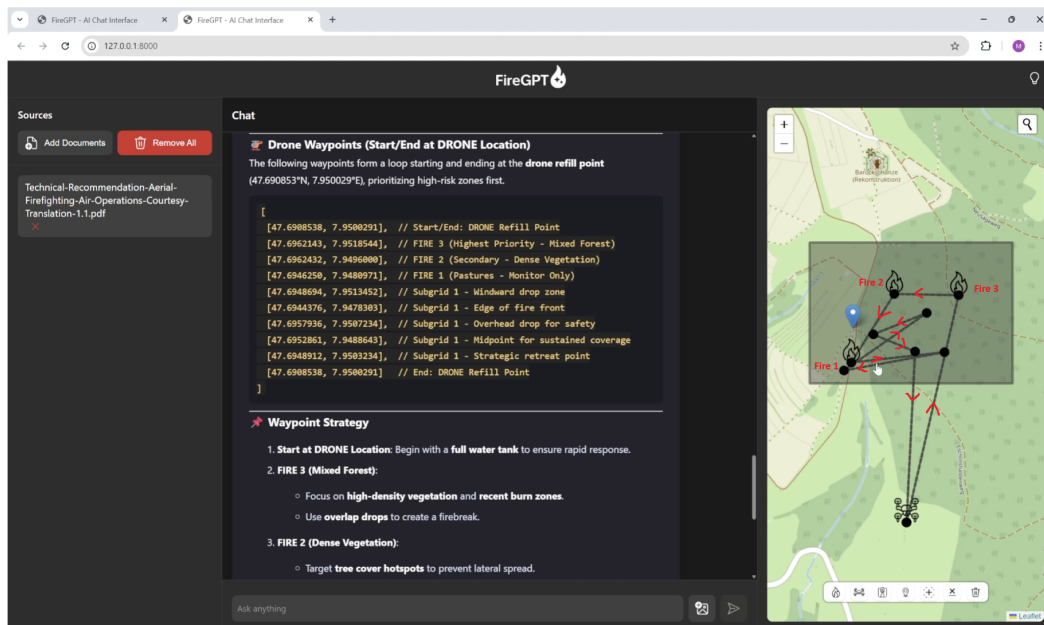


Figure 15: Optimized Drone Waypoint Route Generation usecase

Use Case 1: Generating Attack Plans from SOPs

In this scenario, FireGPT was provided with several Standard Operating Procedure (SOP) PDF documents. The goal was to assess its ability to synthesize a comprehensive attack plan based on these guidelines. The system successfully ingested and processed the textual information from the PDFs. When queried, FireGPT generated an attack plan that was consistent with the provided SOPs, demonstrating its capability to ground responses in authoritative text. This highlights the effectiveness of the agentic RAG strategy, where the LLM dynamically called retrieval tools to access relevant SOP passages.

Use Case 2: Assessing Attack Plans from Images

This use case involved uploading an image containing an attack plan (e.g., a hand-drawn sketch or a satellite image with annotations). FireGPT's Qwen2.5-VL vision-language model processed the image, converting it into a structured caption that listed detected objects, annotated routes, and spatial relations. The ReAct agent then fused this visual information with retrieved SOP passages and GeoJSON POIs in a single reasoning pass to provide a critique or assessment of the plan's coverage gaps and suggest doctrine-aligned guidance. As shown in Figure 14, the system provided detailed feedback on SOP compliance, critical considerations, and country-specific notes, demonstrating its ability to translate visual input into actionable insights. This scenario showcased FireGPT's seamless workflow where users can drop a satellite tile or photo of a paper attack plan into chat and immediately receive doctrine-aligned guidance, safe waypoint suggestions, or a critique of the plan's coverage gaps—all without preprocessing or manual transcription.

Use Case 3: Optimized Drone Waypoint Route Generation

For this critical operational task, users provided existing coordinates, and FireGPT was tasked with evaluating these and generating an optimized drone waypoint route. The system leveraged its real-time situational awareness capabilities. Users can sketch an approximate fire-region bounding box and drop POIs (e.g., "must protect hospital"). The agent then grids that box, invokes the fire-assessment tool for per-cell wind, fuel, and risk metrics, and updates advice as the geometry changes. This allowed the agent to plan tool calls accordingly: retrieving SOP passages relevant to high-risk cell types and ranking candidate drone waypoints that avoid high-flame-length cells and honor "must protect" POIs. The output was a structured JSON payload of latitude-longitude waypoints for drones, which the map layer rendered instantly, allowing operators to see a safe flight path rather than parse a paragraph under stress. This is depicted in Figure 15. This demonstrates FireGPT's ability to produce actionable outputs that translate insight into immediate action, supporting rapid "what-if" iterations.

7.1. Observations

Several key observations and design takeaways emerged during the development and containerization of the FireGPT.

1. **Modular Architecture:** The separation of concerns into different Python modules (`main.py`, `data_fetcher.py`, `fire_model.py`, `poi_fetcher.py`) and distinct Docker services

(ami-backend, ami-mcp, ollama) proved to be a robust architectural choice. This modularity simplifies development, testing, and maintenance.

2. **Importance of Data Validation:** Using Pydantic models for API request and response validation was crucial for ensuring data integrity and providing clear error messages for invalid inputs. This is especially important for a system that deals with complex geospatial data.
3. **Efficient Model Management:** The strategy of using volume mounts for large Hugging Face and Ollama models, rather than embedding them in Docker images, is a critical practice. It significantly reduces image size, speeds up build times, and allows for independent updates of models without rebuilding the application images.
4. **Inter-Service Communication:** Docker Compose simplifies the networking between services. By defining service names and using environment variables, the backend can reliably communicate with the MCP and Ollama services, which is essential for the overall workflow.
5. **Handling External Service Dependencies:** The system's reliance on external APIs like Google Earth Engine and Overpass highlights the need for robust error handling and fallback mechanisms. For instance, the GFS data fetcher was designed to try a broader time range if data for a specific forecast hour is unavailable.
6. **LLMs Agentic / Tool Calling Abilities:** Benchmarking Mistral-7B-Instruct-v0.3, Ministral-8B-Instruct, Pixtral-12B, Qwen2.5-VL-7B, and Qwen3-8B, Qwen3-32B showed the newest Qwen models to be clear leaders, displaying stronger agentic behaviour, more coherent chain-of-thought reasoning, and more precise tool-call composition than the other open-weight options.
7. **Model size and Quantization:** Experiments with 4-bit checkpoints for Qwen3-32B and Qwen3-8B suggest that the 8-B model, when quantised to 4 bits, offers the optimal balance between inference speed and accuracy. A 4-bit 8-B configuration consumes only around 6GB of VRAM, allowing deployment on standard consumer-grade GPUs.

7.2. Trends

FireGPT demonstrated consistently strong performance on most tasks, thanks to precise document retrieval and dependable image description generation. Difficulties appear briefly in waypoint planning, where the agent may sometimes enter prolonged reasoning loops, exhaust the context window through repeated tool calls, and sometimes produce waypoint json that do not conform to the required schema or fail to appear altogether. Reissuing the prompt usually succeeds, suggesting that the core capability is sound but sensitive to context management and prompt framing.

7.3. Interpretation of the Results in Relation to Goals

The points below match each original goal with what we actually observed in testing and state whether the goal was fully met.

1. **Clean, Workflow-centric UI:** Users could chat, upload PDFs, images, and interact directly on a live map all in a single window(see fig. 3).

Status: Achieved

2. **Modular architecture:** All external capabilities (retrieval, fire-assessment) operate as independent MCP tools. LLMs (Qwen3-8B) are served locally via Ollama, and components (e.g., vector stores, models) can be swapped without changes to other modules.

Status: Achieved

3. **Evidence-grounded answers (no hallucination):** From results, the agent always retrieved and cited the relevant SOP text before giving guidance, avoiding unsupported “model guesses.”

Status: Achieved

4. **Vision Understanding:** In trials, uploaded images were auto-captioned and combined with textual data, producing useful, doctrine-based feedback.

Status: Achieved

5. **Real-time situational awareness:** Bounding boxes and POIs can be dynamically adjusted. Whenever users redrew the bounding box area, the system re-scored each grid cell, feeding iterative replans.

Status: Achieved

6. **Actionable output, not just text:** The system returned structured JSON way points that appeared immediately on the map, saving operators from manual parsing.

Status: Achieved

7. **Multilingual reach, edge deployability:** Open-weight models run on a 16GB GPU and worked well in English and German tests, but broader language trials and low-bandwidth drills are still pending. In future, unifying Qwen2.5VL and Qwen3 into a stronger VLM like Qwen3VL (to be released soon) would decrease the VRAM requirement significantly from 16GB to 12GB.

Status: Partially Achieved

7.4. Critical Assessment of the Results and Assumptions

The project’s design relied on six key premises. Five proved sound during evaluation. The second (Internet connectivity) was only partially met, creating a notable gap.

1. **Field hardware:** A single 16 GB-VRAM GPU workstation was indeed enough. Qwen3-8B and Qwen2.5-VL hit latency targets on GPU and stayed within memory limits.

Status: VALID

2. **Internet connectivity:** We assumed full offline operation, but the deployed build needs web access for two services:

- *Map tiles* (basemap and overlays).
- *Fire-assessment API* that queries Google Earth Engine and OpenStreetMap to compute near-real-time danger scores.

Status: PARTIALLY VALID

3. **Legal access to documents:** All SOPs and manuals used in tests were cleared and properly cited; no content-access issues arose.

Status: VALID

4. **Language coverage:** English and German outputs were fluent; other languages were not formally tested, so quality there remains an open question rather than a failure.

Status: VALID, with limits

5. **Image characteristics** All incoming JPEG/PNG images were captioned correctly; no format-related errors were observed.

Status: VALID

6. **User expertise** Operators with basic incident-command skills navigated chat, map, and drawing tools effectively, needing no LLM-specific training.

Status: VALID

Take-away: Hardware, legal, and multimodal assumptions held firm. The sole friction point is reliance on live Internet data.

8. Conclusion

The FireGPT successfully delivered a modern, web-based, multi-modal application tailored for wildfire management, addressing the complex demands of real-time firefighting operations. By integrating advanced Agentic Retrieval-Augmented Generation (RAG) techniques, vision language processing, and geospatial analytics, FireGPT provides a robust platform for generating attack plans, assessing visual inputs, and optimizing drone way-point routes. The system's ability to process diverse inputs (textual SOPs, imagery, and geospatial annotations) while producing contextually grounded and actionable outputs demonstrates its potential to enhance decision making in high-stakes scenarios.

9. Summary of the Results

FireGPT, a multimodal decision-support system for wildfire management, was evaluated across three use cases: generating attack plans from SOPs, assessing plans from images, and optimizing drone waypoint routes.

- **Attack Plans from SOPs:** FireGPT used its RAG pipeline with a tiered document store to generate doctrine-aligned attack plans, reducing mental load.
- **Assessing Plans from Images:** Using Qwen2.5-VL, FireGPT processed visual inputs like sketches and satellite imagery, providing actionable feedback.
- **Drone Waypoint Routes:** FireGPT generated optimized drone routes using geospatial annotations and real-time fire risk assessments.

The intuitive interface, modular architecture (MCP, Ollama etc), no hallucination, vision understanding, real-time situational awareness, actionable output, edge deployability, and multilingual potential ensured operational resilience and scalability, meeting project goals.

10. Future Work

While the current system provides a solid foundation for fire danger assessment, several areas could be enhanced with additional time and resources.

Upcoming efforts will add persistent memory so that users can seamlessly resume earlier sessions, integrate Qwen-2.5-VL, Qwen-3-8B and, once released, Qwen-3-VL to enable image ingestion alongside semantic retrieval, evaluate multilingual performance to guarantee global applicability, pre-cache map tiles and deploy an on-device, lightweight fire-risk model, with a coarser static dataset as an offline fallback, undertake extensive scenario-based testing to surface and remedy bugs, ensuring robust operation across diverse conditions.

11. Comments to the Group Work Experience

We maintained a balanced communication rhythm: quick questions and minor issues were resolved in a WhatsApp group, a weekly Discord call for progress reviews, GitHub for all code, and a focused in-person session every other Wednesday to tackle tougher issues. Presentations were co-edited in Office365, and the final report was drafted collaboratively in Overleaf (TUM ShareLaTeX). Clear tasks and goals kept everyone aligned, so work from every contributor fit together smoothly.

References

- [1] U.S. Department of Agriculture Forest Service, “Maafs operating plan,” 2025, accessed: 2025-07-24. [Online]. Available: <https://www.fs.usda.gov/sites/default/files/maafs-operating-plan.pdf>
- [2] —, “Standards for airtanker operations,” 2022, accessed: 2025-07-24. [Online]. Available: <https://www.fs.usda.gov/sites/default/files/2022-11/Standards-for-Airtanker-Ops.pdf>
- [3] U.S. Department of Agriculture Forest Service and U.S. Department of the Interior Bureau of Land Management, “Federal aerial firefighting: Assessing safety and effectiveness,” 2002, accessed: 2025-07-24. [Online]. Available: https://www.fs.usda.gov/sites/default/files/media_wysiwyg/fed.aerial.ff._assessing.safety.effectivenesss.brp_.2002.pdf
- [4] Los Angeles County Fire Department, “After action review of the woolsey fire incident,” 2018, accessed: 2025-07-24. [Online]. Available: <https://file.lacounty.gov/SDSInter/bos/supdocs/144968.pdf>
- [5] California Governor’s Office of Emergency Services, “After action report: 2016 fire season,” 2016, accessed: 2025-07-24. [Online]. Available: https://www.caloes.ca.gov/wp-content/uploads/Preparedness/Documents/2016-Fires-AAR-Approved_508-JK.pdf
- [6] National Aerial Firefighting Centre, “National aerial firefighting strategy 2021–2026,” 2021, accessed: 2025-07-24. [Online]. Available: https://www.nafc.org.au/wp-content/uploads/2021/07/NAFF_Strategy_Webversion_2021-07-30_v1.1.1.pdf
- [7] A. T. S. Bureau, “A safety analysis of aerial firefighting occurrences in australia: July 2000 to march 2020,” 2020, accessed: 2025-07-24. [Online]. Available: https://www.atsb.gov.au/sites/default/files/media/5777923/ar-2020-022_final.pdf
- [8] M. Plucinski, J. Gould, G. McCarthy, and J. Hollis, “The effectiveness and efficiency of aerial firefighting in australia, part 1,” Bushfire Cooperative Research Centre, Technical Report A0701, 2007, accessed: 2025-07-24. [Online]. Available: https://cdpsdocs.state.co.us/coe/Website/Data_Repository/The%20Effectiveness%20and%20Efficiency%20of%20Aerial%20Firefighting%20in%20Australia_Plucinski%2C%20et%20al..pdf
- [9] M. Plucinski, “Review of aerial suppression effectiveness research literature,” 2025, accessed: 2025-07-24. [Online]. Available: https://www.naturalhazards.com.au/sites/default/files/2025-03/Review-of-aerial-suppression-effectiveness-literature_final.pdf
- [10] Myndigheten för samhällsskydd och beredskap (MSB), “Vägledning i skogsbrandsläckning - 3:e utgåvan 2022,” 2022, accessed: 2025-07-24. [Online]. Available: <https://rib.msb.se/filer/pdf/30038.pdf>

- [11] European Commission – Copernicus Emergency Management Service, “Effis annual fire report 2023,” 2023, accessed: 2025-07-24. [Online]. Available: https://data.effis.emergency.copernicus.eu/effis/reports-and-publications/annual-fire-reports/Annual_Report_2023.pdf
- [12] Canadian Interagency Forest Fire Centre, “Wildfire aviation challenges workshop report 2023,” 2023, accessed: 2025-07-24. [Online]. Available: https://ciffc.ca/sites/default/files/2024-04/CIFFC_Aviation_Challenges_Workshop_Report_2023.pdf
- [13] National Wildfire Coordinating Group, “Standards for aerial supervision (pms 505),” 2025, accessed: 2025-07-24. [Online]. Available: <https://fs-prod-nwcg.s3.us-gov-west-1.amazonaws.com/s3fs-public/publication/pms505.pdf>
- [14] —, “Standards for airtanker base operations (pms 508),” 2025, accessed: 2025-07-24. [Online]. Available: <https://fs-prod-nwcg.s3.us-gov-west-1.amazonaws.com/s3fs-public/publication/pms508.pdf>
- [15] —, “Standards for helicopter operations (pms 510),” 2019, accessed: 2025-07-24. [Online]. Available: <https://www.nifc.gov/sites/default/files/blm/aviation/BLMhelicopters/pms510.pdf>
- [16] Deutscher Feuerwehrverband (DFV), “Technical recommendation – aerial firefighting air operations (courtesy translation),” 2022, accessed: 2025-07-24. [Online]. Available: <https://www.feuerwehrverband.de/app/uploads/2022/05/Technical-Recommendation-Aerial-Firefighting-Air-Operations-Courtesy-Translation-1.1.pdf>
- [17] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 9, 2024.
- [18] OpenAI, “Gpt-4 prompting guide,” https://cookbook.openai.com/examples/gpt4-1_prompting_guide, 2024, accessed: 2025-07-22.

Appendices

A. Source Code and Build Instructions

Repository

The full code is hosted on GitHub:

<https://github.com/mumerabbasi/FireGPT>

Docker Compose build

- Clone the repository and run `run_server.sh`. This script downloads every model, puts them in the correct path, and launches all containers.
- For additional details on Dockerfile builds, consult `EXECUTION.md` in the repository.